ACTIVIDAD PRÁCTICA GUIADA — INTEGRACIÓN CONTINUA, AUTOMATIZACIÓN DE PRUEBAS Y ANÁLISIS DE CALIDAD DEL CÓDIGO

1. OBJETIVO DE LA ACTIVIDAD

En esta actividad se preparará un entorno profesional de integración continua para el proyecto Biblioteca Digital API.

Se partirá de una aplicación que ya dispone de pruebas unitarias, pruebas de integración y pruebas de extremo a extremo (E2E). El trabajo consistirá en preparar un repositorio GitHub y automatizar la ejecución de dichas pruebas, además de integrar un sistema de análisis de calidad del código.

Se construirán **dos workflows independientes de GitHub Actions**:

WORKFLOW 1 — Automatización de pruebas

Será responsable de compilar el proyecto y ejecutar automáticamente las pruebas unitarias, de integración y E2E.

WORKFLOW 2 — Informe de calidad

Será responsable de analizar el código mediante SonarQube Cloud, generar e importar informes de cobertura y comprobar el resultado del Quality Gate.

No se proporcionan archivos YAML preparados. Cada alumno deberá crear la estructura de GitHub Actions, configurar los procesos y comprobar su funcionamiento.

Además, se experimentará con diferentes situaciones: ejecuciones correctas, errores de compilación, pruebas fallidas, ramas, Pull Requests, artefactos, cobertura, problemas de calidad y protección de la rama principal.

1.1. Resultado final esperado

Al finalizar la actividad, el repositorio de GitHub deberá tener una estructura similar a esta:

```
BibliotecaDigital-NombreAlumno/
│
├── .github/
│   └── workflows/
│       ├── tests.yml
│       └── sonar-quality.yml
│
├── src/
│   └── Biblioteca.Api/
│
├── tests/
│   ├── Biblioteca.UnitTests/
│   ├── Biblioteca.IntegrationTests/
│   └── Biblioteca.E2ETests/
│
```

```
├── BibliotecaDigitalE2E.sln
├── .gitignore
└── README.md
```

El repositorio deberá permitir que cualquier cambio enviado a GitHub pueda provocar la ejecución automática de las pruebas.

También deberá realizarse un análisis de calidad que permita consultar los indicadores del proyecto desde SonarQube Cloud.

---

2. CONOCER EL PROYECTO DE PARTIDA

Antes de trabajar con GitHub Actions, es necesario conocer qué se está automatizando.

La aplicación representa una API REST para gestionar el catálogo de libros y los préstamos de una biblioteca.

Está desarrollada con ASP.NET Core y .NET 10. Los datos se almacenan en memoria, por lo que no es necesario configurar una base de datos.

2.1. Estructura del proyecto

El ZIP proporcionado contiene:

```
BibliotecaDigitalE2E/
│
├── src/
│   └── Biblioteca.Api/
│
├── tests/
│   ├── Biblioteca.UnitTests/
│   ├── Biblioteca.IntegrationTests/
│   └── Biblioteca.E2ETests/
│
├── BibliotecaDigitalE2E.sln
└── .gitignore
```

Los tres proyectos de pruebas son independientes y utilizan xUnit.

Biblioteca.UnitTests

Contiene pruebas unitarias de la lógica de negocio de `LibraryService`. Utiliza Moq para simular las dependencias.

Biblioteca.IntegrationTests

Comprueba el comportamiento de los controladores y la API mediante `WebApplicationFactory` y `HttpClient`. No necesita arrancar manualmente un servidor independiente.

Biblioteca.E2ETests

Comprueba flujos completos mediante peticiones HTTP reales contra una instancia independiente de la API. Para funcionar necesita que la aplicación esté arrancada previamente.

Estas últimas son pruebas E2E de una API. No se utilizan navegadores ni una interfaz gráfica.

2.2. Pruebas que ya existen

El proyecto parte de 14 pruebas:

Tipo de prueba	Número inicial	Herramientas
Unitarias	5	xUnit y Moq
Integración	7	xUnit, WebApplicationFactory y HttpClient
E2E	2	xUnit y HttpClient
Total	**14**	

Las pruebas unitarias comprueban, entre otros comportamientos, la creación de libros, la duplicidad de ISBN, la eliminación de libros con préstamos activos y las condiciones para crear préstamos.

Las pruebas de integración comprueban respuestas HTTP, consulta de libros, creación de libros, ISBN duplicados, creación de préstamos y errores de consulta.

Las pruebas E2E comprueban dos flujos completos:

- Crear un libro, consultarlo, eliminarlo y comprobar que ya no existe.
- Crear un préstamo y registrar su devolución.

2.3. Endpoints disponibles

La API contiene los siguientes endpoints:

Método	Ruta	Descripción
GET	`/api/health`	Comprueba que la API está funcionando
GET	`/api/books`	Obtiene todos los libros
GET	`/api/books/{id}`	Obtiene un libro por identificador
POST	`/api/books`	Crea un libro
DELETE	`/api/books/{id}`	Elimina un libro
GET	`/api/loans`	Obtiene todos los préstamos
GET	`/api/loans/{id}`	Obtiene un préstamo por identificador
POST	`/api/loans`	Crea un préstamo
PUT	`/api/loans/{id}/return`	Registra la devolución de un préstamo

La aplicación no dispone de un endpoint para modificar los datos de un libro. No se debe intentar automatizar una operación que no está implementada.

2.4. Datos iniciales

La API carga tres libros de ejemplo:

ID	Libro	Ejemplares disponibles
1	Cien años de soledad	3
2	La sombra del viento	2
3	El nombre del viento	0

Estos datos permiten probar distintos escenarios, como realizar un préstamo correctamente o intentar prestar un libro sin ejemplares disponibles.

PARTE 1. PREPARACIÓN LOCAL Y PRIMERAS COMPROBACIONES

3. EXTRAER Y PREPARAR EL PROYECTO

Descargar el ZIP proporcionado y extraerlo en una ubicación de trabajo.

Entrar en la carpeta:

```
BibliotecaDigitalE2E/
```

Es importante situarse en la carpeta que contiene el archivo:

```
BibliotecaDigitalE2E.sln
```

Esta será la raíz del repositorio Git.

3.1. Limpiar los archivos generados

El ZIP puede contener carpetas de compilación o restauración de paquetes.

Estas carpetas no deben subirse a GitHub:

```
bin/
obj/
TestResults/
```

Eliminar las carpetas `bin` y `obj` que existan dentro de los proyectos.

El archivo `.gitignore` ya contiene reglas para excluir estos elementos cuando se utilice Git.

No eliminar archivos `.cs`, `.csproj`, `.sln` ni ningún otro archivo fuente del proyecto.

3.2. Comprobar que .NET está instalado

Abrir una terminal desde la raíz del proyecto.

Ejecutar:

```
dotnet --info
```

Comprobar que está disponible el SDK de .NET 10.

Si el comando no reconoce `dotnet`, será necesario instalar el SDK antes de continuar.

La instalación se puede realizar desde:

https://dotnet.microsoft.com/download/dotnet/10.0

3.3. Restaurar las dependencias

Ejecutar:

```
dotnet restore BibliotecaDigitalE2E.sln
```

Esto descargará los paquetes NuGet necesarios.

Entre ellos se encuentran:

- xUnit.
- Moq.
- Microsoft.AspNetCore.Mvc.Testing.
- Coverlet.
- Swashbuckle.

3.4. Compilar la solución

Ejecutar:

```
dotnet build BibliotecaDigitalE2E.sln --configuration Release
```

La compilación deberá finalizar correctamente.

La configuración `Release` se utilizará también en los workflows para que las comprobaciones de integración continua se ejecuten de una manera consistente.

3.5. Ejecutar las pruebas unitarias

Ejecutar:

```
dotnet test tests/Biblioteca.UnitTests/Biblioteca.UnitTests.csproj --
configuration Release
```

Comprobar que las cinco pruebas finalizan correctamente.

3.6. Ejecutar las pruebas de integración

Ejecutar:

```
dotnet test
tests/Biblioteca.IntegrationTests/Biblioteca.IntegrationTests.csproj --
configuration Release
```

Comprobar que las siete pruebas finalizan correctamente.

3.7. Ejecutar las pruebas E2E

Estas pruebas necesitan una instancia independiente de la API.

Abrir una primera terminal en la raíz del proyecto y ejecutar:

```
$env:ASPNETCORE_URLS = "http://localhost:5080"

dotnet run --project src/Biblioteca.Api/Biblioteca.Api.csproj --
configuration Release
```

Mantener esta terminal abierta.

La API estará escuchando en:

```
http://localhost:5080
```

Abrir un navegador y comprobar:

```
http://localhost:5080/api/health
```

Debe aparecer una respuesta indicando que el servicio está funcionando.

Abrir una segunda terminal en la raíz del proyecto.

Establecer la dirección que utilizarán las pruebas E2E:

```
$env:E2E_BASE_URL = "http://localhost:5080"
```

Ejecutar:

```
dotnet test tests/Biblioteca.E2ETests/Biblioteca.E2ETests.csproj --
configuration Release
```

Comprobar que los dos flujos E2E finalizan correctamente.

Importante: si la API no está arrancada, las pruebas E2E no podrán conectarse al servicio. No es suficiente con ejecutar `dotnet test` sobre toda la solución sin tener en cuenta este requisito.

PARTE 2. CREACIÓN DEL REPOSITORIO Y CONFIGURACIÓN DE GIT

4. CREAR EL REPOSITORIO EN GITHUB

Acceder a:

https://github.com/

Iniciar sesión y crear un nuevo repositorio mediante la opción:

```
New repository
```

Configurar:

- Nombre: `BibliotecaDigital-NombreAlumno`.
- Descripción: `API Biblioteca Digital con pruebas automatizadas y análisis de calidad`.
- Visibilidad: pública o privada, según las posibilidades de la cuenta y las condiciones de la actividad.

Para utilizar SonarQube Cloud, comprobar que el plan de la cuenta permite analizar el repositorio seleccionado.

No seleccionar las opciones para crear un README, un `.gitignore` o una licencia. El proyecto ya tiene su propio `.gitignore` y el README se creará durante la actividad.

Pulsar:

```
Create repository
```

El repositorio estará creado, pero todavía no tendrá código.

5. CONFIGURAR GIT EN EL ORDENADOR

Volver a la terminal abierta en la raíz de la solución.

5.1. Comprobar Git

Ejecutar:

```
git --version
```

Si Git no está instalado, descargarlo desde:

https://git-scm.com/downloads

5.2. Configurar la identidad

Si todavía no se ha configurado Git en el ordenador:

```
git config --global user.name "Nombre Apellidos"
```

```
git config --global user.email "correo@ejemplo.com"
```

Comprobar la configuración:

```
git config --global --list
```

5.3. Inicializar el repositorio local

Ejecutar:

```
git init
```

Establecer el nombre de la rama principal:

```
git branch -M main
```

Comprobar el estado:

```
git status
```

5.4. Añadir los archivos

Ejecutar:

```
git add .
```

Consultar qué archivos se han añadido:

```
git status
```

Comprobar que no aparecen carpetas `bin` ni `obj`.

Si aparecen, revisar el `.gitignore` y comprobar que no se han añadido previamente al control de versiones.

Crear el primer commit:

```
git commit -m "chore: proyecto inicial Biblioteca Digital"
```

Un commit representa una fotografía del estado de los archivos versionados en ese momento.

5.5. Conectar con GitHub

Copiar la URL HTTPS del repositorio creado.

Será similar a:

```
https://github.com/USUARIO/BibliotecaDigital-NombreAlumno.git
```

Añadir el repositorio remoto:

```
git remote add origin https://github.com/USUARIO/BibliotecaDigital-
NombreAlumno.git
```

Comprobar la conexión:

```
git remote -v
```

Subir el código:

```
git push -u origin main
```

Cuando finalice, actualizar la página del repositorio en GitHub.

Comprobar que aparecen las carpetas `src/`, `tests/`, el archivo `.sln` y el `.gitignore`.

Evidencia 1: captura de pantalla del repositorio de GitHub mostrando la estructura del proyecto y la rama principal.

PARTE 3. PRIMER WORKFLOW: AUTOMATIZACIÓN DE PRUEBAS

6. INTRODUCCIÓN A GITHUB ACTIONS

GitHub Actions permite automatizar operaciones relacionadas con el desarrollo de software.

En lugar de tener que compilar y ejecutar manualmente todas las pruebas cada vez que se modifica el código, GitHub puede hacerlo automáticamente.

Los conceptos principales son:

Workflow: proceso automatizado definido en un archivo YAML.

Event: acontecimiento que inicia un workflow, como enviar cambios al repositorio.

Runner: máquina virtual donde se ejecutan los trabajos.

Job: conjunto de pasos que se ejecutan dentro de un runner.

Step: operación concreta dentro de un job.

Action: componente reutilizable que permite realizar operaciones habituales.

Artifact: archivo o conjunto de archivos que se guarda como resultado de una ejecución.

En esta actividad se utilizarán máquinas virtuales Linux proporcionadas por GitHub.

El ordenador del alumno no tendrá que permanecer encendido para que se ejecuten los workflows.

7. CREAR LA ESTRUCTURA DE WORKFLOWS

Desde la raíz del proyecto, crear la siguiente estructura:

```
.github/
└── workflows/
```

En PowerShell se puede realizar mediante:

```
New-Item -ItemType Directory -Force .github/workflows
```

Dentro de `workflows`, crear el archivo:

```
tests.yml
```

La ruta completa será:

```
.github/workflows/tests.yml
```

Este archivo contendrá el primer workflow.

Los archivos YAML deben respetar correctamente la indentación. Se recomienda utilizar dos espacios por nivel y no emplear tabuladores.

8. CONFIGURAR LOS EVENTOS DEL WORKFLOW

El primer paso es definir qué acontecimientos harán que se ejecute el workflow.

Se utilizarán tres eventos:

push

Se ejecutará cuando se envíen cambios al repositorio.

pull_request

Se ejecutará cuando se abra o actualice una solicitud de integración de cambios dirigida a `main`.

workflow_dispatch

Permitirá iniciar manualmente el proceso desde la interfaz de GitHub.

La configuración será:

```
name: CI - Pruebas automatizadas

on:
  push:

  pull_request:
    branches:
      - main

  workflow_dispatch:
```

Guardar esta configuración y continuar completando el archivo.

9. CREAR EL WORKFLOW DE PRUEBAS

El workflow deberá ejecutar todas las pruebas del proyecto, pero hay que recordar que las pruebas E2E necesitan arrancar la API.

El proceso tendrá que realizar estas operaciones:

1. Descargar el repositorio.
2. Preparar .NET 10.
3. Restaurar dependencias.
4. Compilar la solución.
5. Ejecutar pruebas unitarias.
6. Ejecutar pruebas de integración.
7. Iniciar la API en segundo plano.
8. Comprobar que la API está disponible.
9. Ejecutar pruebas E2E.
10. Detener la API.
11. Guardar los resultados de las pruebas.

El archivo final deberá tener una estructura equivalente a la siguiente.

9.1. Workflow completo

```
name: CI - Pruebas automatizadas

on:
  push:

  pull_request:
    branches:
      - main

  workflow_dispatch:

permissions:
  contents: read

concurrency:
```

```
  group: tests-${{ github.ref }}
  cancel-in-progress: true

jobs:
  tests:
    name: Compilar y ejecutar todas las pruebas
    runs-on: ubuntu-latest
    timeout-minutes: 20

    steps:

      - name: Descargar el repositorio
        uses: actions/checkout@v7

      - name: Preparar .NET 10
        uses: actions/setup-dotnet@v6
        with:
          dotnet-version: '10.0.x'

      - name: Comprobar versión de .NET
        run: dotnet --info

      - name: Restaurar dependencias
        run: dotnet restore BibliotecaDigitalE2E.sln

      - name: Compilar la solución
        run: |
          dotnet build BibliotecaDigitalE2E.sln \
            --configuration Release \
            --no-restore

      - name: Ejecutar pruebas unitarias
        run: |
          dotnet test
tests/Biblioteca.UnitTests/Biblioteca.UnitTests.csproj \
            --configuration Release \
            --no-build \
            --no-restore \
            --logger "trx;LogFileName=unit-tests.trx" \
            --results-directory TestResults/Unit

      - name: Ejecutar pruebas de integración
        run: |
          dotnet test
tests/Biblioteca.IntegrationTests/Biblioteca.IntegrationTests.csproj \
            --configuration Release \
            --no-build \
            --no-restore \
            --logger "trx;LogFileName=integration-tests.trx" \
            --results-directory TestResults/Integration

      - name: Arrancar la API
        run: |
          ASPNETCORE_URLS="http://127.0.0.1:5080" \
          nohup dotnet
src/Biblioteca.Api/bin/Release/net10.0/Biblioteca.Api.dll \
            > api.log 2>&1 &

          echo $! > api.pid

      - name: Esperar a que la API esté disponible
        run: |
          for attempt in {1..30}; do

            if curl --fail --silent \
              http://127.0.0.1:5080/api/health \
              > /dev/null; then
```

```
              echo "API disponible."
              exit 0
            fi

            echo "Esperando a que arranque la API..."
            sleep 2
          done

          echo "No se ha podido iniciar la API."
          cat api.log
          exit 1

      - name: Ejecutar pruebas E2E
        env:
          E2E_BASE_URL: http://127.0.0.1:5080
        run: |
          dotnet test
tests/Biblioteca.E2ETests/Biblioteca.E2ETests.csproj \
            --configuration Release \
            --no-build \
            --no-restore \
            --logger "trx;LogFileName=e2e-tests.trx" \
            --results-directory TestResults/E2E

      - name: Mostrar registros de la API si hay errores
        if: failure()
        run: cat api.log || true

      - name: Detener la API
        if: always()
        run: |
          if [ -f api.pid ]; then
            kill "$(cat api.pid)" || true
          fi

      - name: Guardar resultados de las pruebas
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: resultados-pruebas
          path: TestResults/
          if-no-files-found: warn
          retention-days: 7
```

10. ENTENDER EL WORKFLOW PASO A PASO

No basta con que el archivo funcione. Es necesario comprender sus partes.

10.1. Nombre

```
name: CI - Pruebas automatizadas
```

Es el nombre que aparecerá en la pestaña Actions.

10.2. Eventos

```
on:
  push:
  pull_request:
    branches:
      - main
  workflow_dispatch:
```

Permite definir cuándo se inicia el proceso.

Al no limitar las ramas del evento `push`, se ejecutará al enviar cambios a cualquier rama.

El evento `pull_request` se limita a solicitudes dirigidas a `main`.

10.3. Permisos

```
permissions:
  contents: read
```

El workflow solo necesita permisos de lectura para descargar el código.

Conviene no conceder permisos innecesarios.

10.4. Concurrencia

```
concurrency:
  group: tests-${{ github.ref }}
  cancel-in-progress: true
```

Permite cancelar una ejecución anterior de la misma referencia cuando se inicia otra.

Por ejemplo, si se realizan tres commits muy seguidos, no tiene sentido mantener ejecutándose varias versiones antiguas del mismo proceso.

10.5. Jobs

```
jobs:
  tests:
```

Los jobs representan unidades de trabajo del workflow.

En esta primera versión se utiliza un único job que realiza todas las operaciones de manera secuencial.

Más adelante se ampliará para separar los tipos de pruebas en diferentes jobs.

10.6. Runner

```
runs-on: ubuntu-latest
```

Indica que se utilizará una máquina virtual Linux de GitHub.

Cada ejecución dispone de un entorno de trabajo independiente.

10.7. Descargar el repositorio

```
uses: actions/checkout@v7
```

Esta acción descarga los archivos de GitHub en el runner.

Sin ella, los siguientes comandos no podrían acceder al código fuente.

10.8. Preparar .NET 10

```
uses: actions/setup-dotnet@v6
with:
  dotnet-version: '10.0.x'
```

Instala y configura el SDK de .NET 10.

10.9. Restaurar y compilar

La restauración obtiene los paquetes NuGet necesarios.

La compilación comprueba que los proyectos pueden convertirse correctamente en ensamblados ejecutables.

Se utiliza `Release` para que las ejecuciones automatizadas mantengan una configuración uniforme.

10.10. Ejecutar las pruebas unitarias

Se ejecuta exclusivamente:

```
tests/Biblioteca.UnitTests/
```

El parámetro `--no-build` indica que no se vuelva a compilar el proyecto porque ya se ha compilado la solución en un paso anterior.

El parámetro `--no-restore` evita repetir la restauración.

El logger TRX genera un informe con el resultado de las pruebas.

10.11. Ejecutar las pruebas de integración

Se ejecuta:

```
tests/Biblioteca.IntegrationTests/
```

Estas pruebas utilizan `WebApplicationFactory`, por lo que no necesitan iniciar manualmente una instancia de la API.

10.12. Arrancar la API

El proyecto E2E no utiliza un servidor de pruebas interno.

Necesita una API independiente escuchando en el puerto 5080.

El workflow ejecuta el ensamblado generado por la compilación anterior y lo deja funcionando en segundo plano.

Se utiliza:

```
ASPNETCORE_URLS
```

para indicar la dirección en la que debe escuchar ASP.NET Core.

Los registros se guardan en:

```
api.log
```

El identificador del proceso se guarda en:

```
api.pid
```

Esto permite detenerlo cuando las pruebas hayan terminado.

10.13. Comprobar la disponibilidad

No basta con arrancar un proceso. Hay que comprobar que realmente responde.

El workflow realiza hasta treinta intentos, esperando dos segundos entre cada uno.

Se utiliza:

```
/api/health
```

Si la API no responde, el workflow se detendrá y mostrará los registros.

10.14. Ejecutar E2E

Se establece la variable:

```
E2E_BASE_URL=http://127.0.0.1:5080
```

El proyecto E2E utiliza esta variable para construir la dirección del `HttpClient`.

De esta manera, los tests pueden ejecutarse tanto en el ordenador del alumno como en GitHub Actions sin modificar el código fuente.

10.15. Detener la API

```
if: always()
```

Indica que este paso debe ejecutarse incluso cuando algún paso anterior haya fallado.

Así se evita dejar el proceso de la API ejecutándose innecesariamente en el runner.

10.16. Guardar resultados

```
uses: actions/upload-artifact@v7
```

Permite almacenar los archivos de resultados para descargarlos posteriormente desde GitHub.

Se utilizarán los ficheros TRX generados durante las pruebas.

La retención se establece en siete días para esta actividad.

PARTE 4. PROBAR EL WORKFLOW DE AUTOMATIZACIÓN

11. SUBIR EL PRIMER WORKFLOW

Una vez terminado `tests.yml`, guardar el archivo.

Desde la raíz del proyecto ejecutar:

```
git status
```

Añadir los cambios:

```
git add .
```

Crear un commit:

```
git commit -m "ci: crear workflow de pruebas automatizadas"
```

Subirlo a GitHub:

```
git push
```

Acceder al repositorio y abrir:

```
Actions
```

Deberá aparecer el workflow:

```
CI - Pruebas automatizadas
```

Abrir la ejecución.

Comprobar los diferentes pasos y observar cómo se ejecutan de forma secuencial.

Una ejecución correcta deberá finalizar con un indicador verde.

Si aparece un error, acceder al paso que ha fallado y examinar los registros.

12. EJECUTAR EL WORKFLOW MANUALMENTE

El workflow también permite iniciarse sin hacer un nuevo commit.

Acceder a:

```
Actions
```

Seleccionar:

```
CI - Pruebas automatizadas
```

Pulsar:

```
Run workflow
```

Seleccionar la rama `main` y ejecutar.

Comprobar que empieza una nueva ejecución.

Responder en el informe:

1. ¿Qué evento permite iniciar manualmente un workflow?
2. ¿En qué apartado de GitHub se puede ejecutar?
3. ¿Se necesita crear un nuevo commit para iniciar este proceso?
4. ¿Qué diferencia hay entre `push` y `workflow_dispatch`?

Evidencia 2: captura de una ejecución manual del workflow.

13. PROBAR UNA EJECUCIÓN MEDIANTE PUSH

Modificar el archivo `.gitignore` o cualquier otro archivo de documentación.

Añadir, por ejemplo, una línea de comentario indicando la finalidad del proyecto.

Guardar el archivo.

Ejecutar:

```
git add .
git commit -m "docs: actualizar documentación"
git push
```

Acceder a GitHub Actions.

Comprobar que el evento `push` ha iniciado automáticamente una nueva ejecución.

Identificar en los detalles de la ejecución:

- El evento que la ha provocado.
- La rama.
- El commit.
- El usuario que ha realizado el cambio.
- El resultado de los jobs.

14. PROVOCAR UN ERROR EN UNA PRUEBA UNITARIA

Ahora se va a comprobar cómo reacciona la integración continua ante una prueba fallida.

Abrir:

```
tests/Biblioteca.UnitTests/LibraryServiceTests.cs
```

Seleccionar una aserción existente.

Modificar temporalmente el resultado esperado para que la prueba falle.

Por ejemplo, cambiar una comparación por un valor incorrecto.

Ejecutar la prueba localmente:

```
dotnet test tests/Biblioteca.UnitTests/Biblioteca.UnitTests.csproj
```

Comprobar que falla.

Subir el cambio a GitHub:

```
git add .
git commit -m "test: provocar fallo controlado"
git push
```

Acceder a Actions y examinar la nueva ejecución.

Comprobar:

- Qué paso ha fallado.
- Qué información aparece en los registros.
- Qué error devuelve xUnit.
- Si se ejecutan los pasos posteriores.
- Si GitHub muestra la ejecución en rojo.
- Si se han guardado los resultados de las pruebas.

Importante: por defecto, cuando falla un paso, los siguientes pasos normales no se ejecutan. Los pasos con condiciones como `if: always()` sí pueden ejecutarse.

Restaurar inmediatamente la aserción original.

Subir la corrección y comprobar que el workflow vuelve a finalizar correctamente.

15. PROVOCAR UN ERROR DE COMPILACIÓN

Se va a realizar otra prueba de fallo.

Modificar temporalmente un archivo C# de la API introduciendo un error de sintaxis.

Por ejemplo, eliminar una llave necesaria o escribir incorrectamente una instrucción.

Subir el cambio.

Comprobar la ejecución del workflow.

Responder:

1. ¿Qué paso detecta el error?
2. ¿Se llegan a ejecutar las pruebas unitarias?
3. ¿Se llegan a ejecutar las pruebas de integración?
4. ¿Se llegan a ejecutar las pruebas E2E?
5. ¿Por qué es conveniente compilar antes de ejecutar las pruebas?

Restaurar el archivo y volver a subirlo.

16. PROVOCAR UN FALLO EN UNA PRUEBA E2E

Se va a comprobar que el workflow puede detectar errores en los flujos completos.

Abrir:

`src/Biblioteca.Api/Controllers/BooksController.cs`

Localizar el método `Delete`.

Modificar temporalmente el comportamiento para que la operación no elimine realmente el libro.

Por ejemplo, se puede hacer que el método devuelva una respuesta correcta sin realizar la llamada al servicio que elimina el libro.

Subir los cambios.

Comprobar qué ocurre.

Las pruebas unitarias y de integración existentes podrían seguir pasando, puesto que no tienen una prueba que compruebe ese comportamiento concreto de eliminación desde la API.

La prueba E2E `CompleteBookLifecycle_CreateReadAndDelete` deberá detectar que el libro sigue existiendo después de la operación.

Examinar el resultado.

Restaurar el código original y subir la corrección.

Evidencia 3: captura de una ejecución fallida por una prueba y otra ejecución posterior correcta.

17. DESCARGAR LOS ARTEFACTOS

Abrir una ejecución del workflow que haya generado resultados.

Localizar el apartado:

`Artifacts`

Descargar:

`resultados-pruebas`

Descomprimir el archivo.

Comprobar que aparecen los informes TRX.

Los archivos TRX contienen información sobre las pruebas ejecutadas y sus resultados.

Examinar su contenido mediante un editor de texto o una herramienta compatible.

Responder:

- ¿Qué contiene un fichero TRX?
- ¿Por qué se guarda como artefacto?
- ¿Qué ventaja tiene poder descargar los resultados después de que termine el runner?
- ¿Qué ocurriría si no se configurase el paso de subida de artefactos?

Evidencia 4: captura del apartado Artifacts y del contenido descargado.

PARTE 5. RAMAS, PULL REQUESTS Y PROTECCIÓN DE MAIN

18. CREAR UNA RAMA DE TRABAJO

En un entorno profesional no siempre se trabaja directamente sobre la rama principal.

Se suelen crear ramas para desarrollar cambios de forma independiente.

Desde la terminal:

```
git checkout -b feature/documentacion
```

Comprobar la rama actual:

```
git branch
```

Modificar el archivo `README.md` cuando se cree en la parte de documentación, o añadir un archivo `DOCUMENTACION.md` temporalmente.

Subir los cambios:

```
git add .
git commit -m "docs: ampliar documentacion del proyecto"
git push -u origin feature/documentacion
```

Acceder a GitHub.

Comprobar que existe la nueva rama.

Comprobar también que el evento `push` ha iniciado el workflow de pruebas sobre esa rama.

19. CREAR UN PULL REQUEST

Un Pull Request permite solicitar que los cambios de una rama se integren en otra.

Acceder a GitHub y seleccionar la opción para crear un Pull Request.

Configurar:

```
Base: main
Compare: feature/documentacion
```

Añadir un título descriptivo:

```
Documentación del proyecto Biblioteca Digital
```

Crear el Pull Request.

GitHub deberá ejecutar el workflow configurado para el evento `pull_request`.

Esperar a que termine.

Comprobar qué información aparece en el Pull Request:

- Estado de las comprobaciones.
- Ejecuciones de GitHub Actions.
- Commits incluidos.
- Archivos modificados.
- Diferencias respecto a `main`.

Si el proceso finaliza correctamente, completar la integración mediante la opción correspondiente de GitHub.

Volver a la rama principal en el ordenador:

```
git checkout main
```

Actualizar el contenido local:

```
git pull
```

Comprobar que los cambios de la rama ya aparecen en `main`.

20. COMPROBAR UN PULL REQUEST CON ERRORES

Crear otra rama:

```
git checkout -b feature/pruebas-fallidas
```

Modificar temporalmente una prueba para que falle.

Subir el cambio:

```
git add .
git commit -m "test: comprobar bloqueo por pruebas fallidas"
git push -u origin feature/pruebas-fallidas
```

Crear un Pull Request dirigido a `main`.

Comprobar que GitHub informa de que las comprobaciones han fallado.

No integrar el cambio.

Restaurar la prueba, crear un nuevo commit y subir la corrección:

```
git add .
git commit -m "test: corregir prueba"
git push
```

Comprobar que GitHub vuelve a ejecutar el workflow sobre el Pull Request.

Cuando termine correctamente, completar la integración.

21. PROTEGER LA RAMA MAIN

Se va a configurar una regla para evitar que se integren cambios que no hayan superado las comprobaciones.

Acceder a:

```
Settings → Rules → Rulesets
```

Crear un nuevo conjunto de reglas para ramas.

Configurar la regla para que afecte a:

`main`

Siempre que las opciones de la cuenta lo permitan, activar:

- Exigir que los cambios se integren mediante Pull Request.
- Exigir que se completen las comprobaciones seleccionadas.
- Evitar integraciones cuando los controles obligatorios no hayan finalizado correctamente.

Seleccionar el workflow de pruebas como comprobación obligatoria.

Guardar la regla.

Crear un Pull Request de prueba y comprobar cómo afecta la protección de la rama principal a la integración.

Si alguna opción no está disponible por las características del repositorio o del plan de GitHub, documentar la limitación.

Responder:

1. ¿Qué ventaja tiene proteger la rama `main`?
2. ¿Qué diferencia hay entre un push y un Pull Request?
3. ¿Por qué interesa ejecutar las pruebas antes de integrar cambios?
4. ¿Qué sucede cuando una comprobación obligatoria falla?

PARTE 6. CONFIGURAR SONARQUBE CLOUD

22. ¿QUÉ ES SONARQUBE CLOUD?

SonarQube Cloud es una plataforma que permite analizar la calidad y seguridad del código fuente.

A diferencia de las pruebas automatizadas, que ejecutan el programa y comprueban su comportamiento, el análisis de SonarQube examina el código para detectar problemas potenciales.

Entre otras cosas, permite localizar:

- Bugs potenciales.
- Problemas de seguridad.
- Fragmentos que requieren revisión de seguridad.
- Código difícil de mantener.
- Código duplicado.
- Problemas relacionados con la cobertura de pruebas.

En esta actividad, SonarQube Cloud se integrará directamente en GitHub Actions.

El objetivo es que cada ejecución del segundo workflow envíe los resultados al panel de SonarQube Cloud.

23. CREAR UNA CUENTA EN SONARQUBE CLOUD

Acceder a:

https://sonarcloud.io/

Iniciar sesión utilizando GitHub.

Autorizar la integración cuando se solicite.

Crear o seleccionar una organización.

Seguir el asistente de configuración para importar el repositorio de GitHub.

Seleccionar el repositorio:

```
BibliotecaDigital-NombreAlumno
```

Al configurar el método de análisis, seleccionar:

```
With GitHub Actions
```

El asistente mostrará los pasos necesarios para configurar el análisis.

Se debe utilizar el método basado en CI/CD, porque los análisis se ejecutarán mediante el segundo workflow que se creará en esta actividad.

24. CREAR EL PROYECTO EN SONARQUBE CLOUD

Seleccionar el repositorio importado.

Comprobar los datos identificativos del proyecto.

Localizar estos dos valores:

Organization key

Es el identificador de la organización.

Project key

Es el identificador único del proyecto analizado.

Guardar ambos valores para utilizarlos posteriormente en GitHub.

No confundir el nombre visible de la organización con su clave.

25. GENERAR EL TOKEN DE AUTENTICACIÓN

GitHub Actions necesita autenticarse para enviar los resultados a SonarQube Cloud.

Para ello se utilizará un token.

En SonarQube Cloud, acceder a la configuración de la cuenta y localizar la sección de seguridad y tokens.

Generar un token personal.

Copiar el valor generado y guardarlo temporalmente en un lugar seguro.

MUY IMPORTANTE: el token equivale a una credencial privada. No debe escribirse directamente en ningún archivo YAML ni subirse a GitHub.

No incluirlo en capturas de pantalla, documentación ni commits.

26. CONFIGURAR LOS SECRETOS Y LAS VARIABLES EN GITHUB

Volver al repositorio de GitHub.

Acceder a:

```
Settings → Secrets and variables → Actions
```

Se trabajará con dos categorías.

26.1. Crear el secreto

En la pestaña `Secrets`, seleccionar la opción para añadir un secreto.

Crear:

```
Nombre: SONAR_TOKEN
Valor: token generado en SonarQube Cloud
```

Guardar.

Los secretos están pensados para almacenar datos privados.

26.2. Crear las variables

En la pestaña `Variables`, crear estas dos variables:

```
Nombre: SONAR_ORGANIZATION
Valor: organization key
```

Y:

```
Nombre: SONAR_PROJECT_KEY
Valor: project key
```

Guardar ambas.

El repositorio deberá tener:

Secretos:

- `SONAR_TOKEN`

Variables:

- `SONAR_ORGANIZATION`
- `SONAR_PROJECT_KEY`

La diferencia es que el token es confidencial, mientras que los identificadores del proyecto no necesitan tratarse como contraseñas.

El workflow utilizará:

```
${{ secrets.SONAR_TOKEN }}
```

para recuperar el secreto.

Y:

```
${{ vars.SONAR_ORGANIZATION }}
${{ vars.SONAR_PROJECT_KEY }}
```

para recuperar las variables.

Evidencia 5: realizar una captura de la configuración de GitHub donde se aprecien los nombres de las variables y del secreto. No mostrar nunca el valor del token.

PARTE 7. SEGUNDO WORKFLOW: INFORME DE CALIDAD CON SONARQUBE CLOUD

27. CREAR EL ARCHIVO SONAR-QUALITY.YML

Volver al proyecto local.

Dentro de:

```
.github/workflows/
```

Crear el archivo:

```
sonar-quality.yml
```

Su ruta completa será:

```
.github/workflows/sonar-quality.yml
```

Este segundo workflow será independiente del primero.

Su objetivo será:

1. Descargar el código.
2. Preparar .NET 10.
3. Instalar SonarScanner.
4. Restaurar las dependencias.
5. Iniciar el análisis de SonarQube.
6. Compilar la solución.
7. Ejecutar las pruebas unitarias generando cobertura.
8. Ejecutar las pruebas de integración generando cobertura.
9. Arrancar la API.
10. Ejecutar las pruebas E2E.
11. Finalizar el análisis.
12. Esperar el resultado del Quality Gate.
13. Guardar los informes.

28. ENTENDER LA COBERTURA DE CÓDIGO

Antes de escribir el workflow hay que comprender qué se va a medir.

La cobertura de código indica qué parte del código ha sido ejecutada durante las pruebas.

Por ejemplo, si el informe muestra un 70 % de cobertura de líneas, significa que aproximadamente el 70 % de las líneas contabilizadas han sido ejecutadas al menos una vez durante las pruebas.

Esto no significa que el 70 % del software esté libre de errores.

Tampoco significa que todas las funcionalidades estén correctamente probadas.

Se utilizará Coverlet, que ya está incluido como dependencia de los proyectos de pruebas.

Coverlet generará los informes de cobertura en formato OpenCover.

SonarQube Cloud no genera por sí mismo estos informes: los recibe e incorpora a su análisis.

28.1. ¿Por qué no se utiliza la cobertura E2E para medir la API?

Las pruebas E2E se conectan a una instancia independiente de la API mediante HTTP.

El código de la API se está ejecutando en otro proceso, separado del proceso de pruebas.

Por tanto, la cobertura recogida desde el proceso de tests E2E no representa correctamente qué líneas internas de la API se han ejecutado.

En esta actividad se utilizará la cobertura de las pruebas unitarias y de integración para medir el código de la aplicación.

Las pruebas E2E seguirán ejecutándose como parte del proceso de análisis, pero su finalidad principal será comprobar el comportamiento externo de la API.

29. CREAR EL WORKFLOW DE SONARQUBE CLOUD

El archivo final deberá tener una estructura equivalente a la siguiente:

```
name: SonarQube Cloud - Análisis de calidad

on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main
    types:
      - opened
      - synchronize
      - reopened

  workflow_dispatch:
```

```
permissions:
  contents: read
  pull-requests: read

concurrency:
  group: sonar-${{ github.ref }}
  cancel-in-progress: true

jobs:
  quality:
    name: Compilación, cobertura y análisis
    runs-on: ubuntu-latest
    timeout-minutes: 30

    steps:

      - name: Descargar el repositorio
        uses: actions/checkout@v7
        with:
          fetch-depth: 0

      - name: Preparar .NET 10
        uses: actions/setup-dotnet@v6
        with:
          dotnet-version: '10.0.x'

      - name: Comprobar versión de .NET
        run: dotnet --info

      - name: Instalar SonarScanner para .NET
        run: |
          dotnet tool install --global dotnet-sonarscanner
          echo "$HOME/.dotnet/tools" >> "$GITHUB_PATH"

      - name: Restaurar dependencias
        run: dotnet restore BibliotecaDigitalE2E.sln

      - name: Iniciar análisis de SonarQube Cloud
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_ORGANIZATION: ${{ vars.SONAR_ORGANIZATION }}
          SONAR_PROJECT_KEY: ${{ vars.SONAR_PROJECT_KEY }}
        run: |
          dotnet sonarscanner begin \
            /k:"$SONAR_PROJECT_KEY" \
            /o:"$SONAR_ORGANIZATION" \
            /d:sonar.token="$SONAR_TOKEN" \
            /d:sonar.host.url="https://sonarcloud.io" \

/d:sonar.cs.opencover.reportsPaths="**/coverage.opencover.xml" \
            /d:sonar.qualitygate.wait=true

      - name: Compilar solución para análisis
        run: |
          dotnet build BibliotecaDigitalE2E.sln \
            --configuration Release \
            --no-restore \
            --no-incremental \
            --disable-build-servers

      - name: Pruebas unitarias y cobertura
        run: |
          dotnet test
tests/Biblioteca.UnitTests/Biblioteca.UnitTests.csproj \
            --configuration Release \
            --no-build \
            --no-restore \
```

```
            --collect:"XPlat Code Coverage;Format=opencover" \
            --results-directory TestResults/Unit \
            --logger "trx;LogFileName=unit-tests.trx"

      - name: Pruebas de integración y cobertura
        run: |
          dotnet test
tests/Biblioteca.IntegrationTests/Biblioteca.IntegrationTests.csproj \
            --configuration Release \
            --no-build \
            --no-restore \
            --collect:"XPlat Code Coverage;Format=opencover" \
            --results-directory TestResults/Integration \
            --logger "trx;LogFileName=integration-tests.trx"

      - name: Arrancar API para las pruebas E2E
        run: |
          ASPNETCORE_URLS="http://127.0.0.1:5080" \
          nohup dotnet
src/Biblioteca.Api/bin/Release/net10.0/Biblioteca.Api.dll \
            > api.log 2>&1 &

          echo $! > api.pid

      - name: Esperar disponibilidad de la API
        run: |
          for attempt in {1..30}; do

            if curl --fail --silent \
              http://127.0.0.1:5080/api/health \
              > /dev/null; then

              echo "API disponible."
              exit 0
            fi

            echo "Esperando a que arranque la API..."
            sleep 2
          done

          echo "No se ha podido iniciar la API."
          cat api.log
          exit 1

      - name: Ejecutar pruebas E2E
        env:
          E2E_BASE_URL: http://127.0.0.1:5080
        run: |
          dotnet test
tests/Biblioteca.E2ETests/Biblioteca.E2ETests.csproj \
            --configuration Release \
            --no-build \
            --no-restore \
            --logger "trx;LogFileName=e2e-tests.trx" \
            --results-directory TestResults/E2E

      - name: Detener la API
        if: always()
        run: |
          if [ -f api.pid ]; then
            kill "$(cat api.pid)" || true
          fi

      - name: Finalizar análisis de SonarQube Cloud
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
        run: |
```

```
        dotnet sonarscanner end \
          /d:sonar.token="$SONAR_TOKEN"

    - name: Guardar informes generados
      if: always()
      uses: actions/upload-artifact@v7
      with:
        name: informes-calidad-y-cobertura
        path: TestResults/
        if-no-files-found: warn
        retention-days: 7
```

30. COMPRENDER EL WORKFLOW DE CALIDAD

30.1. ¿Por qué se descarga todo el historial de Git?

Se utiliza:

```
fetch-depth: 0
```

Esto permite disponer del historial completo del repositorio.

El historial facilita que SonarQube Cloud identifique correctamente las ramas, los Pull Requests y los cambios analizados.

30.2. Instalar SonarScanner

Se utiliza:

```
dotnet tool install --global dotnet-sonarscanner
```

Esto instala la herramienta de análisis para proyectos .NET.

Posteriormente se incorpora su directorio al PATH del runner:

```
$HOME/.dotnet/tools
```

Así se puede utilizar el comando:

```
dotnet sonarscanner
```

El escáner .NET moderno puede obtener automáticamente el entorno Java necesario para ejecutar el análisis; no es necesario instalar un JDK adicional para esta configuración.

30.3. Iniciar el análisis

El comando:

```
dotnet sonarscanner begin
```

inicia el análisis.

Se le proporcionan los datos necesarios para identificar y autenticar el proyecto.

Clave del proyecto:

```
SONAR_PROJECT_KEY
```

Organización:

`SONAR_ORGANIZATION`

Token:

`SONAR_TOKEN`

También se establece la ubicación de los informes de cobertura:

`sonar.cs.opencover.reportsPaths`

Y se configura la espera del Quality Gate:

`sonar.qualitygate.wait=true`

El análisis permanece abierto durante la compilación y la ejecución de las pruebas.

30.4. Compilar durante el análisis

El escáner debe estar activo cuando se compila la solución.

Por eso el comando `dotnet build` aparece después de `dotnet sonarscanner begin`.

Se utiliza:

`--no-incremental`

para forzar una compilación completa.

También se utiliza:

`--disable-build-servers`

para evitar que los servidores de compilación reutilizados interfieran en el análisis.

30.5. Generar informes de cobertura

En las pruebas unitarias y de integración aparece:

`--collect:"XPlat Code Coverage;Format=opencover"`

Este parámetro permite que Coverlet genere un informe de cobertura durante la ejecución de las pruebas.

Se generan informes independientes para:

- `Biblioteca.UnitTests`.
- `Biblioteca.IntegrationTests`.

Los informes se guardan dentro de las carpetas `TestResults`.

El formato OpenCover es compatible con la importación de cobertura de C# en SonarQube Cloud.

30.6. Importar los informes

En el inicio del análisis se ha configurado:

```
sonar.cs.opencover.reportsPaths
```

Esta propiedad indica al escáner dónde debe localizar los informes OpenCover.

La expresión:

```
**/coverage.opencover.xml
```

permite buscar los ficheros generados en las carpetas de resultados.

Si los informes no existen o no se encuentran, SonarQube Cloud no podrá mostrar correctamente la cobertura.

30.7. Finalizar el análisis

El comando:

```
dotnet sonarscanner end
```

finaliza el proceso y envía los datos recogidos a SonarQube Cloud.

Se utiliza el mismo token que se había utilizado al iniciar el análisis.

30.8. Esperar el Quality Gate

La configuración:

```
sonar.qualitygate.wait=true
```

indica que el workflow debe esperar el resultado del Quality Gate.

Esto es muy importante.

Puede suceder que:

- Todas las pruebas hayan pasado.
- La compilación haya finalizado correctamente.
- El análisis estático haya terminado.
- SonarQube Cloud haya detectado problemas que impiden superar el Quality Gate.

En ese caso, el workflow puede terminar en rojo aunque las pruebas automatizadas hayan pasado.

No se deben confundir los resultados de las pruebas con el resultado del Quality Gate.

PARTE 8. EJECUTAR SONARQUBE CLOUD Y ANALIZAR SUS RESULTADOS

31. SUBIR EL SEGUNDO WORKFLOW

Una vez terminado `sonar-quality.yml`, guardar el archivo.

Desde la terminal:

```
git add .
```

Crear un commit:

```
git commit -m "ci: añadir análisis de calidad con SonarQube Cloud"
```

Subir los cambios:

```
git push
```

Acceder a GitHub Actions.

Deberán aparecer dos workflows independientes:

```
CI - Pruebas automatizadas

SonarQube Cloud - Análisis de calidad
```

Comprobar que los dos se ejecutan cuando corresponde.

El workflow de pruebas se ejecuta cuando se realiza un push o se abre un Pull Request dirigido a `main`, además de permitir la ejecución manual.

El workflow de SonarQube Cloud se ejecuta al realizar un push a `main`, al abrir o actualizar un Pull Request dirigido a `main` y mediante ejecución manual.

Si el repositorio utiliza una rama principal con otro nombre, adaptar las referencias correspondientes.

32. COMPROBAR EL ANÁLISIS EN SONARQUBE CLOUD

Acceder a:

https://sonarcloud.io/

Abrir el proyecto importado.

Comprobar que aparece un análisis correspondiente al commit que se acaba de subir.

Examinar las diferentes secciones del panel.

32.1. Quality Gate

El Quality Gate indica si el proyecto supera las condiciones de calidad que tiene configuradas.

Puede mostrar un resultado satisfactorio o un resultado que requiere correcciones.

Examinar las condiciones que se están aplicando.

Responder:

- ¿Qué es un Quality Gate?
- ¿Qué diferencia hay entre superar las pruebas y superar el Quality Gate?
- ¿Puede terminar correctamente la compilación y fallar el Quality Gate?

32.2. Bugs

Son problemas que pueden provocar un comportamiento incorrecto del programa.

Examinar los posibles problemas detectados.

Si no aparece ninguno, indicarlo en el informe.

32.3. Vulnerabilities

Son problemas relacionados con posibles vulnerabilidades de seguridad.

Examinar qué información aporta el panel.

No se debe interpretar que cualquier código marcado por el análisis constituye necesariamente una vulnerabilidad explotable: hay que estudiar el problema concreto.

32.4. Security Hotspots

Son fragmentos de código que requieren revisión desde el punto de vista de la seguridad.

No todos son vulnerabilidades confirmadas.

Se debe revisar la explicación y comprender qué comportamiento recomienda comprobar SonarQube.

32.5. Code Smells

Son problemas relacionados con la estructura o mantenibilidad del código.

No siempre provocan errores de ejecución.

Sin embargo, pueden dificultar futuras modificaciones y aumentar el coste del mantenimiento.

Examinar los posibles Code Smells.

32.6. Coverage

Muestra el porcentaje de código cubierto por las pruebas.

Comprobar si SonarQube Cloud ha importado correctamente los informes OpenCover.

Si no aparece la cobertura, revisar:

1. Si los proyectos de pruebas ejecutaron Coverlet.
2. Si se generaron ficheros `coverage.opencover.xml`.
3. Si los informes se guardaron en `TestResults`.
4. Si la propiedad `sonar.cs.opencover.reportsPaths` está bien configurada.
5. Si los informes se generaron antes de finalizar el análisis.

No asumir que la cobertura es del 0 % si los informes no se han importado: puede tratarse de un problema de configuración.

32.7. Duplications

Esta métrica muestra información sobre código duplicado.

Examinar si existen fragmentos que se repiten innecesariamente.

Explicar por qué la duplicación puede dificultar el mantenimiento.

32.8. Maintainability, Reliability y Security

Examinar los indicadores de mantenibilidad, fiabilidad y seguridad.

Anotar qué representa cada uno y qué información permite conocer sobre el estado del proyecto.

Evidencia 6: capturas de las secciones principales del panel de SonarQube Cloud.

PARTE 9. AMPLIAR LAS PRUEBAS Y MEJORAR LA COBERTURA

33. AMPLIAR LAS PRUEBAS UNITARIAS

El proyecto parte de cinco pruebas unitarias.

Se deben incorporar, como mínimo, tres pruebas unitarias adicionales para ampliar los comportamientos cubiertos.

Se pueden trabajar situaciones como:

- Intentar crear un préstamo de un libro inexistente.
- Intentar devolver un préstamo que no existe.
- Intentar devolver un préstamo que ya está devuelto.
- Crear un préstamo cuando existen ejemplares disponibles.
- Comprobar que no se permite eliminar un libro con préstamos activos.
- Comprobar que se puede eliminar un libro sin préstamos activos.

Utilizar Moq para simular las dependencias cuando sea necesario.

Las pruebas deben comprobar tanto el resultado devuelto por el servicio como las interacciones relevantes con los repositorios.

Ejecutar todas las pruebas unitarias y comprobar que pasan.

34. AMPLIAR LAS PRUEBAS DE INTEGRACIÓN

El proyecto parte de siete pruebas de integración.

Incorporar, como mínimo, tres pruebas adicionales que comprueben el comportamiento de la API mediante HTTP.

Se pueden trabajar situaciones como:

- Crear un libro con datos no válidos y comprobar que se devuelve un error de validación.
- Crear un préstamo para un libro inexistente.
- Intentar realizar un préstamo cuando no hay ejemplares disponibles.
- Devolver un préstamo existente.
- Intentar devolver un préstamo dos veces.
- Eliminar un libro que tiene un préstamo activo.
- Comprobar que una operación de eliminación correcta devuelve el código HTTP esperado.

En estas pruebas se deben comprobar los códigos de respuesta HTTP y, cuando corresponda, el contenido de las respuestas.

35. AMPLIAR LAS PRUEBAS E2E

El proyecto parte de dos pruebas E2E.

Incorporar, como mínimo, dos pruebas o flujos E2E adicionales.

Los nuevos flujos deben utilizar HTTP real contra la API.

Se pueden implementar escenarios como:

Flujo de préstamo completo

1. Consultar un libro del catálogo.
2. Comprobar que tiene ejemplares disponibles.
3. Crear un préstamo.
4. Consultar el préstamo generado.
5. Comprobar que el libro tiene un ejemplar menos.
6. Registrar la devolución.
7. Comprobar que el préstamo aparece como devuelto.
8. Comprobar que el libro recupera el ejemplar.
9. Intentar devolver de nuevo el préstamo y comprobar que la API rechaza la operación.

Flujo de eliminación

1. Crear un libro nuevo.
2. Crear un préstamo para ese libro.
3. Intentar eliminar el libro mientras el préstamo está activo.
4. Comprobar que la API rechaza la eliminación.
5. Devolver el préstamo.
6. Eliminar el libro.
7. Comprobar que ya no existe.

Utilizar datos únicos cuando sea necesario para evitar colisiones con otros tests.

Los tests deben poder ejecutarse repetidamente sin depender de que una ejecución anterior haya dejado datos preparados.

36. COMPARAR LA COBERTURA ANTES Y DESPUÉS

Cuando se hayan ampliado las pruebas:

1. Ejecutar localmente los proyectos de pruebas.
2. Comprobar que todas las pruebas funcionan.
3. Subir los cambios a GitHub.
4. Esperar a que finalice el workflow de calidad.
5. Acceder a SonarQube Cloud.
6. Consultar la nueva cobertura.
7. Compararla con el resultado anterior.

En el informe se debe explicar:

- Qué nuevas pruebas se han añadido.
- Qué comportamientos se cubren ahora.
- Si ha aumentado la cobertura.
- Qué diferencias existen entre cobertura de líneas y cobertura de condiciones, cuando se muestren ambas métricas.
- Por qué tener un porcentaje elevado de cobertura no garantiza que el software esté libre de errores.

Evidencia 7: capturas de las nuevas pruebas y comparación de la cobertura antes y después.
